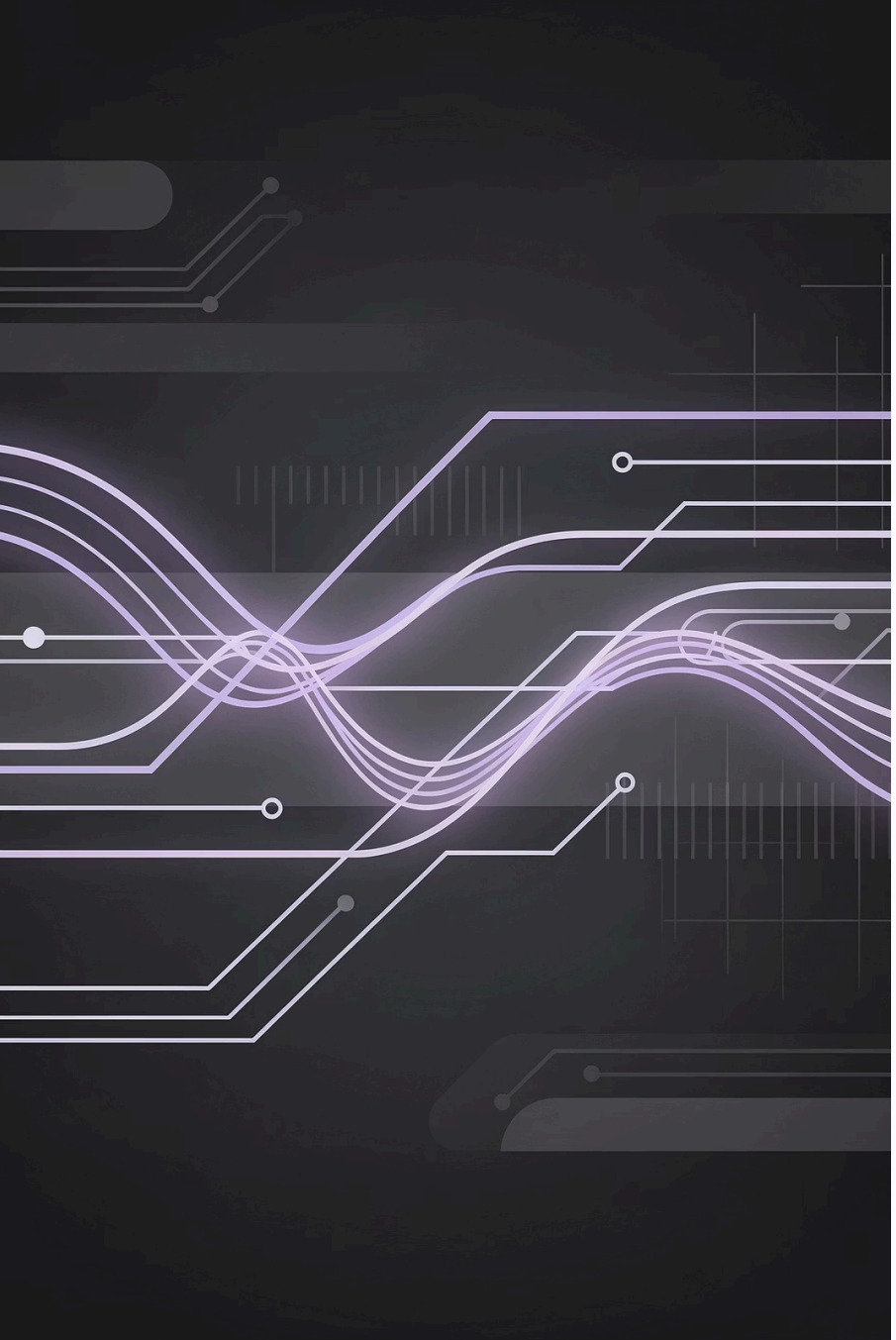


Structured Streaming + Kafka

Building real-time data pipelines with Apache Spark's Structured Streaming and Apache Kafka — from first principles to production-grade architecture.

APACHE SPARK | APACHE KAFKA | REAL-TIME PIPELINES

What is Structured Streaming?

Stream Processing, Simplified

Structured Streaming is Spark's high-level, declarative API for processing continuous, unbounded data streams. Introduced in Spark 2.0, it treats a live data stream as an ever-growing table — abstracting away the complexity of traditional streaming frameworks.

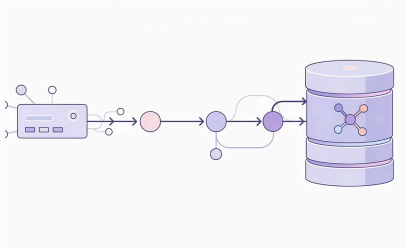
Unified Batch & Streaming API

The defining advantage of Structured Streaming is that it exposes the **same DataFrame and Dataset API** used in batch processing. Engineers familiar with Spark SQL or PySpark can write streaming logic without learning a separate paradigm.

- Use `spark.readStream` instead of `spark.read`
- Apply standard transformations: `filter`, `groupBy`, `join`
- Spark handles micro-batch execution and state management internally

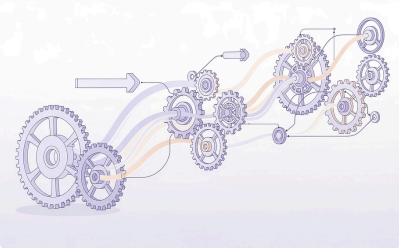
Key Concepts

Every Structured Streaming pipeline is built from three fundamental building blocks. Understanding each layer is essential before writing a single line of streaming code.



Source

The **input** to your streaming query. Spark reads continuously from a source — Kafka topics, file directories, socket connections, or Delta tables. Sources are fault-tolerant and replay-capable, ensuring no data is lost on restart.



Transformation

The **logic layer** applied to the incoming stream. This includes filtering, aggregating, joining with static or streaming datasets, and applying custom UDFs — all expressed using familiar DataFrame operations.

Sink

The **output destination** where processed results are written. Supported sinks include Kafka, Delta Lake, Parquet files, JDBC databases, and the console. Each sink defines how and where results land after each micro-batch.

Reading Streams with `spark.readStream`

The `spark.readStream` API is the entry point for all streaming queries in Structured Streaming. It returns a `DataStreamReader` that you configure with a format, connection options, and an optional schema.

Spark supports a variety of built-in stream sources out of the box:

Kafka

Subscribe to one or more topics by name or pattern. Spark tracks offsets automatically per partition.

File Source

Monitor a directory for new files (JSON, CSV, Parquet, Avro). New files are picked up automatically each trigger.

Delta Lake

Stream changes from a Delta table using the transaction log. Ideal for lakehouse architectures.

Example: Reading from Kafka

```
df = spark.readStream \
 .format("kafka") \
 .option(
 "kafka.bootstrap.servers",
 "broker:9092"
 ) \
 .option(
 "subscribe",
 "events-topic"
 ) \
 .option(
 "startingOffsets",
 "latest"
 ) \
 .load()

# Deserialise the value column
from pyspark.sql.functions import col

parsed = df.selectExpr(
 "CAST(key AS STRING)",
 "CAST(value AS STRING)",
 "timestamp"
)
```

Writing Streams with `.writeStream`

Example: Writing to Delta Lake

```
query = parsed \
  .writeStream \
  .format("delta") \
  .outputMode("append") \
  .option(
    "checkpointLocation",
    "/checkpoints/events"
  ) \
  .trigger(
    processingTime="10 seconds"
  ) \
  .start(
    "/delta/events"
  )

query.awaitTermination()
```

The `.writeStream` API configures how processed results are materialised. It is the counterpart to `readStream` and must be called to actually execute a streaming query.

Key configuration options to understand:

→ `.format()`

Specifies the output sink — `delta`, `kafka`, `parquet`, `console`, or a custom sink.

→ `.outputMode()`

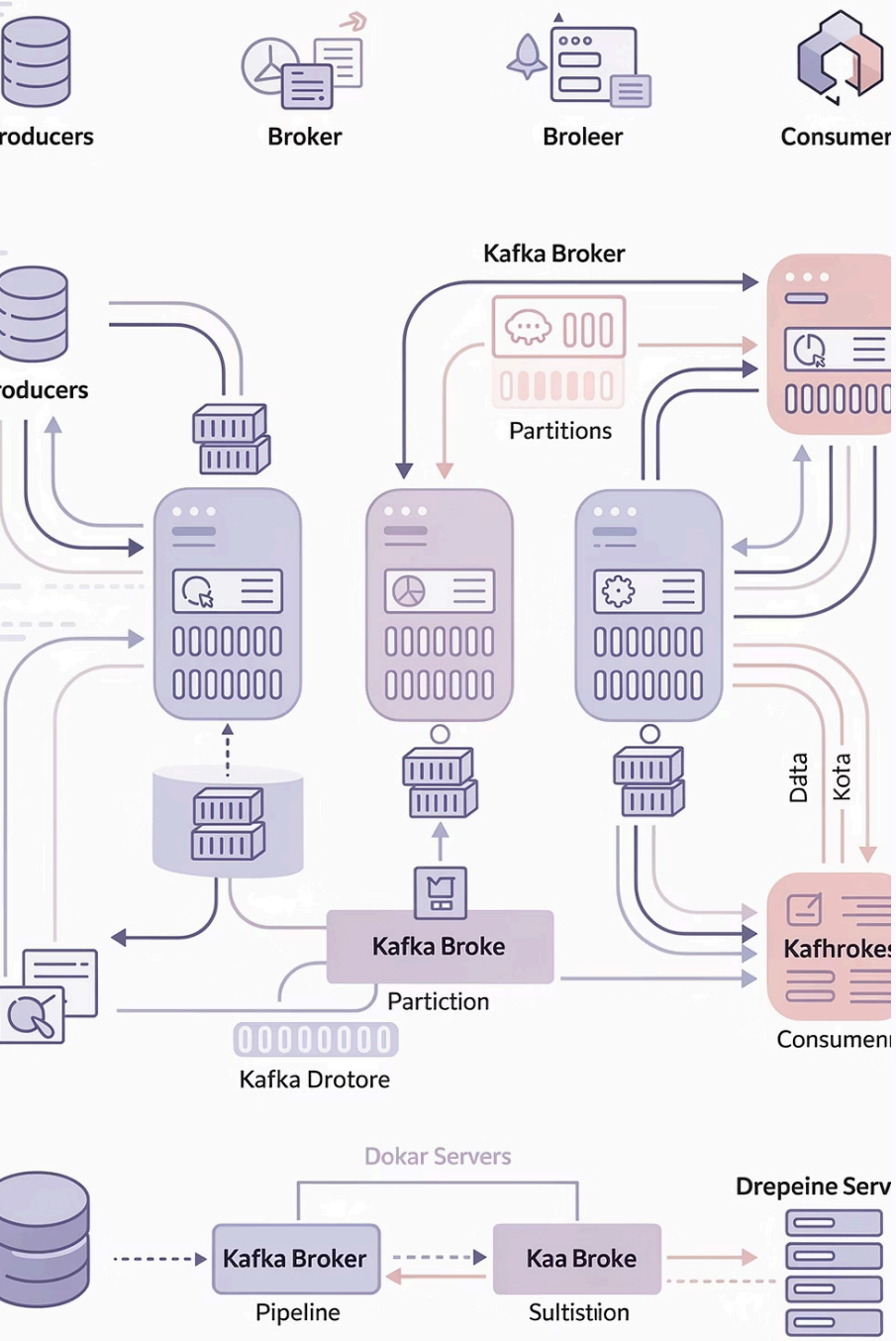
Controls which rows are written each trigger: `append`, `update`, or `complete`.

→ `.trigger()`

Sets the processing cadence — fixed interval, once, or continuous (low-latency) mode.

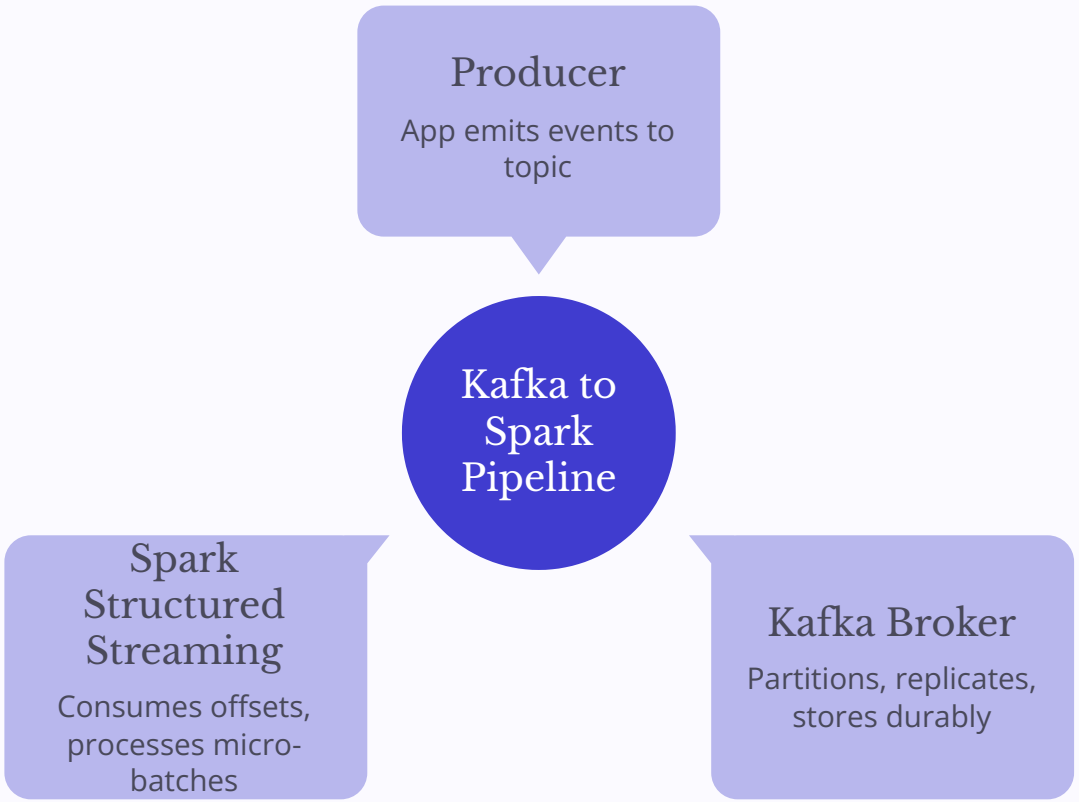
→ `.start()`

Launches the streaming query and returns a `StreamingQuery` handle for monitoring and control.



Kafka Integration

Apache Kafka acts as the distributed, fault-tolerant backbone of most real-time Spark pipelines. The data flow follows a clear producer-to-consumer pattern:



Spark reads from Kafka by tracking **per-partition offsets** in its checkpoint directory. On restart, it resumes exactly where it left off — guaranteeing **exactly-once processing semantics** when combined with an idempotent sink such as Delta Lake.

Checkpointing & Fault Tolerance

Why Checkpointing Matters

Checkpointing is the mechanism by which Structured Streaming achieves **end-to-end fault tolerance**. Without it, a driver failure or cluster restart would result in data loss or reprocessing ambiguity.

Set a checkpoint location on every streaming query — without exception:

```
.option(
  "checkpointLocation",
  "/mnt/checkpoints/my-query"
)
```

What Gets Persisted?

Offsets

The exact Kafka offset (or file position) read at each trigger, preventing re-reads.

Commit Log

A record of successfully completed micro-batches, enabling idempotent restarts.

State Store

Aggregation and join state (e.g. windowed counts) persisted to durable storage for recovery.

Store checkpoints on durable, highly-available storage — HDFS, ADLS, or S3. Never use local disk in production.

Output Modes

Structured Streaming provides three output modes that control **which rows are written to the sink** on each trigger. Choosing the wrong mode for your query type will result in a runtime error or incorrect results.

Append

Only **new rows** added since the last trigger are written. This is the default and most efficient mode.

✅ Suitable for: event logs, raw ingestion, queries without aggregations or with watermarked aggregations that are fully finalised.

Update

Only rows that have **changed since the last trigger** are written. More efficient than Complete for incremental aggregations.

✅ Suitable for: running aggregations (e.g. counts, sums) where only modified keys need to be updated in the sink.

Complete

The **entire result table** is rewritten on every trigger. Guarantees a consistent snapshot of the full aggregation state.

✅ Suitable for: global aggregations with no watermark, or sinks that require a full refresh (e.g. dashboards, caches).

Structured Streaming + Delta Lake

Delta Lake is the recommended sink for production Structured Streaming pipelines. It brings ACID transactions, schema enforcement, and time travel to streaming writes — transforming raw event streams into reliable, queryable tables.

Writing Streams to Delta

- Use `.format("delta")` with `outputMode("append")` for event ingestion
- Delta's transaction log ensures exactly-once writes when checkpointing is enabled
- Concurrent batch reads against the same table are fully supported — no locks required
- Schema evolution can be enabled with `mergeSchema` for flexible ingestion

Incremental Pipelines with Delta

Delta tables serve as both **streaming sinks and streaming sources**, enabling multi-hop lakehouse architectures (Bronze → Silver → Gold):

- **Bronze**: raw events appended directly from Kafka
- **Silver**: cleansed, parsed, deduplicated records streamed from Bronze
- **Gold**: aggregated, business-ready metrics for analytics and dashboards

Let's Build a Real-Time Pipeline

You now have the foundational building blocks. It's time to put them together.

01

Ingest from Kafka

Use `spark.readStream.format("kafka")` to subscribe to a topic and read raw events with offset tracking.

02

Parse & Transform

Deserialise the Kafka value payload, apply schema, filter noise, and enrich with reference data using standard DataFrame operations.

03

Write to Delta

Sink the processed stream to a Delta table with a checkpoint location to guarantee fault-tolerant, exactly-once delivery.

04

Monitor & Iterate

Observe query progress via `query.lastProgress`, set up alerting, and extend the pipeline with Silver and Gold layers.

💡 The patterns covered in this session — readStream, writeStream, Kafka integration, checkpointing, output modes, and Delta sinks — are the complete toolkit for production real-time pipelines on Spark.